

Progetto S2/L5: Analisi del codice "Assistente Virtuale"

Indice

1. Introduzione

2. Corpo / Analisi

2.1. Come dovrebbe funzionare il programma

2.2. Comportamenti imprevisti dell'utente (casi limite)

2.3. Gli errori trovati e come risolverli

3. Conclusione e Raccomandazioni

1. Introduzione

In questo report spiego i risultati dell'analisi che ho fatto sul codice Python "Assistente Virtuale". Come richiesto dalla consegna dell'esercitazione, ho fatto un'analisi "a occhio", cioè ho letto il codice cercando di capire come si comporta e cercando gli sbagli senza farlo partire sul computer.

L'obiettivo di questo documento è spiegare a parole cosa fa il programma, evidenziare cosa succederebbe se un utente scrivesse comandi inaspettati, ed elencare tutti gli errori che ho trovato, spiegando passo dopo passo come fare per risolverli.

2. Corpo / Analisi

2.1 Come dovrebbe funzionare il programma

Leggendo il codice dall'alto verso il basso, si capisce che è un semplice bot da riga di comando. Usa un ciclo infinito (`while True`) per continuare a chiedere all'utente: "Cosa vuoi sapere? ".

Se l'utente risponde scrivendo esattamente "esci", il programma lo saluta e si chiude in automatico. Se l'utente scrive qualsiasi altra cosa, la sua frase viene passata a una funzione chiamata `assistente_virtuale`. Questa funzione usa una serie di controlli (`if`, `elif`, `else`) per vedere se la frase corrisponde a tre domande precise: la data di oggi, l'ora, o il nome del bot. In base alla domanda, la funzione prepara una stringa di risposta e la restituisce, e alla fine il programma la stampa a schermo.

2.2 Comportamenti imprevisti dell'utente (casi limite)

Mettendomi nei panni di un utente che usa il programma, mi sono reso conto che i controlli `if` sono un po' troppo rigidi. Se anche non ci fossero errori nel codice, il programma andrebbe spesso in confusione (rispondendo "Non ho capito") in queste situazioni molto comuni:

- **Maiuscole e minuscole:** Python fa distinzione. Se l'utente scrive "Esci" (con la maiuscola) o "che ore sono?" (con l'iniziale minuscola), il controllo fallisce perché non c'è una corrispondenza esatta con quello che c'è scritto nel codice.
- **Spazi premuti per sbaglio:** Se l'utente preme spazio alla fine di una frase (esempio: "esci "), il programma non lo riconosce a causa dello spazio in più.
- **Punteggiatura:** Se l'utente si dimentica di mettere il punto interrogativo alla fine della domanda, il bot non capisce cosa gli è stato chiesto.

2.3 Gli errori trovati e come risolverli

Guardando attentamente, ho trovato 5 errori veri e propri che impediscono al programma di funzionare. Ecco quali sono e come li sistemerei:

Errore 1: Comando inesistente per la data (Riga 13)

- **Che errore è:** Errore di esecuzione (o di libreria).
- **Cosa succede:** Il codice usa `datetime.datetime.today()`, ma questo comando non esiste in Python. Se ci proviamo, il programma va in crash.
- **Come risolverlo:** Bisogna cancellare quel pezzo e usare il comando giusto, cioè `datetime.date.today()`.

Errore 2: Uso sbagliato del metodo time (Riga 16)

- **Che errore è:** Errore logico / di sintassi con la libreria.
- **Cosa succede:** Il codice prova a prendere l'ora con `datetime.datetime.now().time()`. Però subito dopo usa `.strftime("%H:%M")` per formattarla, e usare quel `.time()` alla fine della riga 16 crea dei problemi tecnici in quel contesto.
- **Come risolverlo:** Basta togliere il pezzo `.time()` e lasciare semplicemente `ora_attuale = datetime.datetime.now()`.

Errore 3: Variabile 'risposta' non definita bene (Righe 21-22)

- **Che errore è:** Errore logico.
- **Cosa succede:** Se l'utente scrive un comando che il bot non capisce, la variabile `risposta` non viene definita in modo sicuro prima di fare `return risposta` alla riga 22. È una cattiva abitudine di programmazione.
- **Come risolverlo:** Bisogna assicurarsi che anche nel caso in cui la domanda non venga riconosciuta, la variabile `risposta` venga creata e riempita con la frase di errore prima di essere restituita dal `return`.

Errore 4: Mancano i due punti nel ciclo (Riga 3)

- **Che errore è:** Errore di sintassi.
- **Cosa succede:** Alla fine di `while True` mancano i due punti (`:`). Senza i due punti, Python si blocca e non fa nemmeno partire il programma.
- **Come risolverlo:** Aggiungere i due punti alla fine della riga, facendola diventare `while True:`.

Errore 5: La funzione è chiamata prima di esistere (Righe 9 e 11)

- **Che errore è:** Errore logico / Nome non definito.
- **Cosa succede:** Alla riga 9 il programma prova a usare la funzione `assistente_virtuale(comando_utente)`. Il problema è che la funzione viene "creata" solo a partire dalla riga 11. Visto che Python legge le righe dall'alto verso il basso, quando arriva alla riga 9 non sa ancora cosa sia questa funzione e va in errore.
- **Come risolverlo:** Bisogna prendere tutto il blocco della funzione (da `def assistente_virtuale` fino al `return risposta`) e spostarlo più in alto, mettendolo prima che inizi il ciclo `while True:`.

(Nota sull'indentazione: Ho notato che alla riga 7 c'è un problema strutturale. Il comando `break` è allineato a sinistra uguale all'`if`. Per funzionare bene, il `break` deve essere spostato un po' più a destra (indentato) in modo che stia dentro l'istruzione `if`).

3. Conclusione e Raccomandazioni

In conclusione, il codice di questa esercitazione non può funzionare così com'è. È bloccato da un errore di sintassi (i due punti mancanti), dalla funzione messa nel posto sbagliato e da comandi scritti male (come quello della `data`).

Raccomandazioni: Oltre a correggere i 5 errori che ho elencato qui sopra, secondo me la cosa migliore da fare per rendere il programma davvero utilizzabile è "pulire" l'input dell'utente prima di fare i controlli `if`. Se usiamo un metodo per le stringhe come `.lower()`, possiamo trasformare tutto quello che l'utente scrive in lettere minuscole in automatico. In questo modo eviteremmo i problemi dei maiuscoli/minuscoli descritti nei "casi limite", rendendo il bot molto più intelligente e meno rigido.